

🔍 Linear Search Algorithm in Python

Definition:

Linear Search (also called Sequential Search) is the simplest search algorithm that checks every element in a list one by one until the desired element is found or the list ends.

✔ Characteristics:

Feature	Description
Time Complexity	$O(n)$ — checks each element once
Space Complexity	$O(1)$ — no extra space needed
Best Case	$O(1)$ — if the target is at the beginning
Worst Case	$O(n)$ — if the target is at the end or not found
Data Requirement	Works on unsorted or sorted lists

❑ How It Works:

- Start from index 0.
 - Compare each element with the target.
 - If match found, return index or element.
 - If not found after checking all elements, return -1 or "Not found".
-

❑ Python Example:

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i # return the index of the found element
    return -1 # target not found

# Example usage
numbers = [5, 3, 7, 1, 9, 2]
target = 1

result = linear_search(numbers, target)
if result != -1:
    print(f"Element found at index: {result}")
else:
    print("Element not found")

# ❑ Output:
# Element found at index: 3
```

✦ When to Use Linear Search?

- When the dataset is small.
 - When the list is unsorted.
 - When performance is not critical.
-

🔍 What is Binary Search?

Binary Search is an efficient algorithm used to find a specific element in a sorted list. It works by dividing the search space in half again and again, reducing the time it takes to find the item.

✓ Prerequisite

- The list **must be sorted** in ascending order (very important).
 - Binary search is more efficient than linear search, especially for large lists.
-

💡 Idea (Analogy)

Suppose you are looking for a number in a book that contains sorted page numbers. Instead of going page by page, you:

- Open the book in the middle.
- Check if it's the page you're looking for.
- If not, decide whether to go left (lower numbers) or right (higher numbers).
- Repeat until you find the page.

☐ This is what binary search does in code.

🔄 Steps (Algorithm)

1. Start with the `left` and `right` pointers at the start and end of the list.
 2. While `left` is less than or equal to `right`:
 - Calculate the middle index.
 - Compare the target with the middle element:
 - If equal → Found the item.
 - If `target < middle` → Search in the **left half**.
 - If `target > middle` → Search in the **right half**.
 3. If the item is not found, return `-1` or "Not Found".
-

☐ Time & Space Complexity

Case	Complexity
------	------------

Best Case	O(1) — Found in the middle
-----------	-----------------------------------

Worst Case	O(log n) — Keep dividing
------------	---------------------------------

Case	Complexity
Space	O(1) — No extra space

🔗 Python Code with Detailed Comments

```
def binary_search(arr, target):
    # Step 1: Initialize pointers
    left = 0           # Start of the list
    right = len(arr) - 1 # End of the list

    # Step 2: Repeat the loop until left crosses right
    while left <= right:
        # Step 3: Find the middle index
        mid = (left + right) // 2

        # Step 4: Check the value at mid
        if arr[mid] == target:
            return mid # Found the target

        # Step 5: Decide where to search next
        elif target < arr[mid]:
            right = mid - 1
        else:
            left = mid + 1

    # Step 6: If not found, return -1
    return -1
```

🔗 Example

```
numbers = [2, 4, 6, 8, 10, 12, 14] # List must be sorted
target = 10

result = binary_search(numbers, target)

if result != -1:
    print(f"Element found at index {result}")
else:
    print("Element not found")
```

Output:

Element found at index 4

☐ Dry Run (Step-by-Step)

```
arr = [2, 4, 6, 8, 10], target = 8
left = 0, right = 4

mid = (0 + 4) // 2 = 2 → arr[mid] = 6

8 > 6 → search right → left = 3
```

```
mid = (3 + 4) // 2 = 3 → arr[mid] = 8  
8 == 8 → FOUND!
```

⊗ What if the list is unsorted?

```
arr = [10, 4, 6, 2, 8] # Not sorted
```

Binary search will give **wrong results**.

✓ Always sort the list first!

★ Bonus: Recursive Version in Python

```
def binary_search_recursive(arr, target, left, right):  
    if left > right:  
        return -1  
  
    mid = (left + right) // 2  
  
    if arr[mid] == target:  
        return mid  
    elif target < arr[mid]:  
        return binary_search_recursive(arr, target, left, mid - 1)  
    else:  
        return binary_search_recursive(arr, target, mid + 1, right)  
  
# Example  
arr = [1, 3, 5, 7, 9]  
print(binary_search_recursive(arr, 7, 0, len(arr) - 1)) # Output: 3
```

✓ Summary

Step	Action
1	Check if the list is sorted
2	Set left and right pointers
3	Loop until <code>left <= right</code>
4	Check middle value
5	Narrow the search (left/right half)
6	Return index if found or -1 if not

♣ What is DFS (Depth First Search)?

DFS is a graph traversal algorithm that starts from a source node and explores as far as possible along each branch before backtracking.

It uses a **stack data structure** (either explicitly or via recursion) and is commonly used in:

- ☐ Solving mazes
 - ☐ Path finding
 - 🔁 Topological sorting
 - 🔁 Cycle detection
 - ☐ Solving puzzles (e.g., Sudoku)
-

★ Key Concepts

- **Graph:** A collection of nodes (vertices) connected by edges.
 - DFS works on both **directed** and **undirected** graphs.
 - DFS can be implemented using:
 - **Recursion**
 - **Explicit Stack**
-

🔄 DFS Process (Step-by-Step)

1. Start at a node (called source).
 2. Mark the node as visited.
 3. For each unvisited neighbor:
 - Recursively apply DFS.
 4. Backtrack when no unvisited neighbor is left.
-

✓ DFS Pseudocode (Recursive)

```
DFS(node) :  
    if node is not visited:  
        mark node as visited  
        for each neighbor of node:  
            DFS(neighbor)
```

☐ DFS vs BFS (Key Differences)

Feature	DFS	BFS
Data Structure	Stack / Recursion	Queue
Explores	As deep as possible	Level by level
Time Complexity	$O(V + E)$	$O(V + E)$
Space Complexity	$O(V)$ (recursion/stack)	$O(V)$ (queue)
Use Case	Pathfinding, Cycle Detection Shortest path in unweighted graphs	

DFS Code in Python

1 Recursive DFS for Adjacency List Graph

```
def dfs(graph, node, visited):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbor in graph[node]:
            dfs(graph, neighbor, visited)

# Example graph (undirected)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

visited = set()
dfs(graph, 'A', visited)
# Output: A B D E F C (order may vary based on graph structure)
```

2 Iterative DFS Using Stack

```
def dfs_iterative(graph, start):
    visited = set()
    stack = [start]

    while stack:
        node = stack.pop()
        if node not in visited:
            print(node, end=" ")
            visited.add(node)
            # Push neighbors in reverse to maintain order
            for neighbor in reversed(graph[node]):
                stack.append(neighbor)

dfs_iterative(graph, 'A')
```

Time and Space Complexity

Let:

- V = number of vertices
- E = number of edges

Operation Complexity

Time $O(V + E)$

Space $O(V)$

☐ Applications of DFS

- ✓ Pathfinding in mazes
 - ✓ Topological Sorting (in DAGs)
 - ✓ Connected Components
 - ✓ Cycle Detection in Graphs
 - ✓ Solving puzzles (e.g., Sudoku)
 - ✓ Web Crawlers (Explore all linked pages)
-

☐ Real-world Analogy

Imagine a maze. DFS keeps going down one path until it can't go any further, then backtracks and tries another.
It's like exploring **every hallway deeply** before trying the next one.

🔍 What is Breadth-First Search (BFS)?

BFS is a graph traversal algorithm. It starts at a starting node (called the **root** in tree structures) and explores all the neighboring nodes at the current depth before moving on to the nodes at the next depth level.

🐘 **Think of it as:**

Waves of exploration — first explore all close nodes, then go deeper step by step.

✓ **It works for:**

- Graphs (connected or disconnected)
 - Trees (BFS in trees = Level Order Traversal)
-

☐ Core Concepts

∞ **Graph Terminology**

- **Node (or Vertex):** A point in the graph.
 - **Edge:** A connection between two nodes.
 - **Neighbor:** A node connected directly to another node.
-

☐ What BFS Needs:

- **Queue:** To process nodes in "First In First Out" (FIFO) order.
 - **Visited Set:** To keep track of already visited nodes (avoid infinite loops).
-

🔑 BFS Strategy (Step-by-step)

1. Start from the source node.
2. Mark it as visited.
3. Put it in a queue.
4. While the queue is not empty:
 - Remove the node from the front.
 - For each unvisited neighbor:

- Mark it as visited
- Add it to the queue

🔗 BFS Implementation in Python (from Scratch)

We'll represent the graph as an **adjacency list** using a dictionary.

Example Graph:

```
A -- B -- D
|      |
C      E
```

Adjacency list:

```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A'],
    'D': ['B'],
    'E': ['B']
}
```

Python Code:

```
from collections import deque # for efficient queue

def bfs(graph, start_node):
    visited = set()           # To keep track of visited nodes
    queue = deque([start_node]) # Start with the starting node

    while queue:
        node = queue.popleft() # Remove node from the front of the queue

        if node not in visited:
            print("Visited:", node)
            visited.add(node)    # Mark the node as visited

            # Add all unvisited neighbors to the queue
            for neighbor in graph[node]:
                if neighbor not in visited:
                    queue.append(neighbor)

# Run the BFS:
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A'],
    'D': ['B'],
    'E': ['B']
}

bfs(graph, 'A')
```

Output:

Visited: A
Visited: B
Visited: C
Visited: D
Visited: E

🔄 How BFS Works (Trace)

- Start at A → Queue: ['B', 'C']
 - Visit B → Queue: ['C', 'D', 'E']
 - Visit C → Queue: ['D', 'E']
 - Visit D → Queue: ['E']
 - Visit E → Queue: [] (done!)
-

❑ When to Use BFS?

- ✓ Find **shortest path** in unweighted graphs
 - ✓ **Level order traversal** of trees
 - ✓ Web crawling
 - ✓ Social networking (shortest connection path)
 - ✓ Maze solving
-

✓ Key Points Summary

Concept	Description
Type	Traversal Algorithm
Data Structure	Queue (FIFO)
Time Complexity	$O(V + E) \rightarrow V$: Vertices, E : Edges
Space Complexity	$O(V)$ for visited set and queue
Use Case	Shortest path, Level order, Connectivity
